



Inspiring Excellence

## **OPTIMIZING CACHE USAGE IN SCIENTIFIC AND MACHINE LEARNING WORKLOADS**

**Submitted by:**

Amir Sakib Saad

22299222

Course Code

CSE340 (COMPUTER ARCHITECTURE)

Section

18

**Submitted to:** SUI

**Department of Computer Science and Engineering**

**BRAC University**

## Introduction

Convolutional Neural Networks require high use of memory bandwidth during training phase and inference while making critical cache performance on both CPUs and GPUs. Generally a multidimensional array with numbers that represent the information of an image, uses the concept of sliding-window operations from data structure and algorithms in order to map and filter a tensor. Although, it's an iterative approach which emphatically requires back to back data accesses from dynamic random access memory (DRAM). However, reading an image means fetching information from the fundamental level of pixels leading to thrashing of cache lines. The aforementioned operations are memory intensive because of reusable pixels and load to DRAM, causing high cost and slow down processes with memory bound complexity, the main focus in computer architecture. DRAMs are slow and require power to hold information which is continuously refreshing [1]. After all optimization of cache usage via software level can significantly enhance the efficiency of energy and power with improved throughput on modern CPUs and GPUs to train a large CNN input.

## Background & Related Work

CNN is often known for its sophisticated data layouts and its reusability architecture. Data layout refers to how the data will be assigned in the memory. To represent a 2 dimensional image, modern architecture uses a 4-D tensor denoted by NCHW/NHCW/CHWN [2]. NHWC is a concept of the segmentation of all channels of a single pixel which perfectly works on GPU whereas NCHW focuses on the same channel with all pixels which perfectly works on CPU. Both of them are the well-known format of data layout which deeply affects the locality. However, choosing a layout by its optimality on AlexNet layers can yield approximately 6.9 times speedup. Even some of the operations in convolutional neural networks that require time for arithmetic and logical operation are now become memory-bound due to array scanning and iterative sliding window operations, an operation that causes latency and energy.

A frequent workflow of data can be achieved through an optimized general matrix multiply convolution that splits a large matrix into small segments and arranges them according to the size of cache memory in CPU which ensures the fetching of data from the cache rather than persistently data retrieval from the memory. Modern technology uses 3 dimensional cache blocking where the cache memory is arranged in layers of input and small segments of outputs, validates the reusability of data and inferences within a short time. This process warrants CPU inference speed between 9% to 25% but only for machine learning models. Overcoming the marginality of deep learning and neural networks requires high level optimization on program loops to ensure the continuous retrieval of data from the memory. That's why gaining knowledge on memory hierarchy is crucial to understanding and solving the perplexity of CNN optimization.

## Methodology & System Design

Loop tiling or cache blocking is something that converts large convolutional loops into blocks or tiles that fit together into the cache. This facilitates multiple computational reusability, and capacitates the increase of cache hit rate. Loop tiling in CNN is advocated to efficiently circulate feature maps [3].

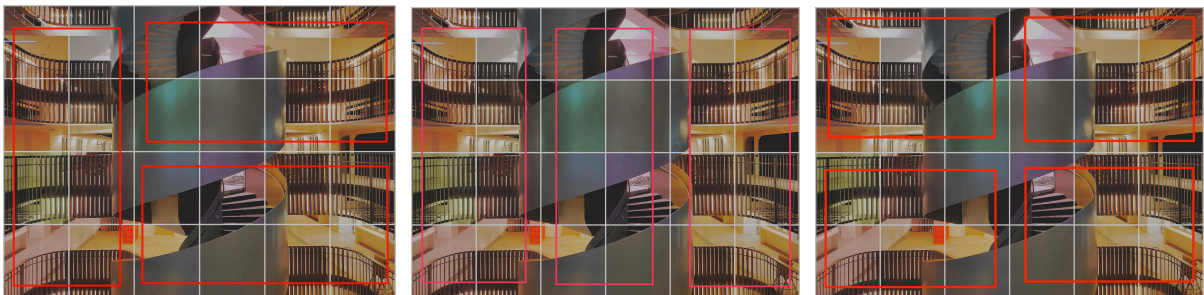


Figure 1: Visual representation of applying blocking on an image.

```

import numpy as np
def Architecture(image, kernel, blockSize):
    Height, Width = image.shape
    Kernel, c = kernel.shape
    outputHeight = Height-Kernel+1
    outputWidth = Width-Kernel+1
    output = np.zeros((outputHeight, outputWidth),dtype=np.float32)
    return Tiling(outputHeight, outputWidth, blockSize, Kernel, kernel, image, output)
def MIN(x, y):
    if x==y:
        return x
    else:
        if x<y:
            return x
        return y
def Tiling(H, W, B, K, k, I, Output):
    for h in range(0,H,B):
        for w in range(0,W,B):
            for blkH in range(h,MIN(h+B,H)):
                for blkW in range(w,MIN(w+B,W)):
                    temp = 0.0
                    for i in range(K):
                        for j in range(K):
                            temp+=I[blkH+i, blkW+j]*k[i,j]
                    Output[blkH, blkW] = temp
    return Output
image = np.random.rand(32,32).astype(np.float32)
kernel = np.random.rand(3,3).astype(np.float32)
block = 16
output = Architecture(image,kernel,block)
print(output)

```

Similar to the Tensorflow's XLA prefetching CNN we can hand drive prefetch instruction to fetch data that have been used in future, camouflaging the latency and cache miss reduction. Model execution and preprocessing on a training step due to prefetching [4]. The entire big data splitted into several chunks and one unit iteration processes one single chunk. The next chunk which is not yet processed is loaded into a different thread. This enables the simultaneous process of chunk processing and chunk pre-loading.

Loop interchange is an optimization process used to improve the pattern of memory accessibility [5]. Loop interchanges refers to the change of sequence of loops for better data accessibility. For example: NCHW format arranges data in a sequential pattern. So running an inner loop on NCHW can fetch indexed data efficiently. On the other hand, NHWC in TensorFlow focuses on GPU and gives an opportunity to transport data from NHWC format to NCHW in order to fit data perfectly on cache.

```

import tensorflow as tf
def NHWC(batch=8, imageHeight=32, imageWidth=32, channel=3, kernel=[3,3], output=16):
    CNNx = tf.random.normal([batch, imageHeight, imageWidth, channel])
    CNNc = tf.random.normal([kernel[0], kernel[1], channel, output])
    CNNy = tf.nn.conv2d(CNNx, CNNc, strides=[1,1,1,1],padding="SAME",data_format="NHWC")
    return CNNy.shape
print(NHWC())

```

## Discussion

Implementing overlaying loops manually and large block size may cause development effort and can diminish the kernel utilization. Due to the demand of FP16 or bfloat16 inputs of Tensor Cores, the accuracy may gradually change. However, jumping from NHWC to NCHW may cost time. Again, Google made XLA optimized the machine learning model in runtime and Apache TVM uses deep learning models ahead of time rather than runtime. Both can streamline tiling while adding compiler time overhead and won't find the optimal schedule for each layer. An on-chip L1 cache consumes less power than that of DRAM access. Reducing cache-miss entails less interaction with DRAM which is

less latency and energy consuming. Approximately 20% of the inference of CNN can be reduced by using optimal cache usages. As an outcome it increases the life cycle of hardware and also saves energy.

In the era of modern science and technology, the use of CNN and image processing is increasing vastly. From the medical sector to security purposes we use CNNs as the core of the algorithm. We use CNN to train models for computer vision, object tracking, medical image analysis where optimization is considered as the higher precedence. Medical image processing needs a huge amount of data on training phase and deep analysis due to its sensitivity accuracy which encourages a high efficient cache optimization system to process tensors data. However, on the other side of the coin, there are some limitations of how many blocks should be created. Sometimes targeting to achieve optimization opportunities, if we reduce the size of block or increase the number of blocks may affect parallelism specially when the input batch size is too large. Local reuse and overall throughput must be maintained simultaneously. Finally, coding and compiler level complexity with balanced hardware resources requires less energy consuming with high performance in CNN.

### **Conclusion**

The significance of cache memory to train a CNN with a large batch is unavoidable. Normal sliding window operation reuse similar information over and over, causing heat generation and low performance issues. However, creating blocks on an image, prefetching and change in data format significantly improve the barrier of CNN optimization, reducing the extra pressure on DRAM using cache. NCHW format can increase speed over 20% to 30% on CPU. Future works like object detection and classification, VLMI training and analysis will be way more efficient and optimized than that of the regular CNN architecture.

## References

- [1] Ngô, C., & B.Natowitz, J. (2017). From microelectronics to nanoelectronics. In *Our Nanotechnology Future* (pp. 136–144). Amsterdam University Press.  
<https://doi.org/10.2307/j.ctv1pncrc8.15>
- [2] *Core Concepts — NVIDIA cuDNN v9.3.0 documentation.* (n.d.).  
<https://docs.nvidia.com/deeplearning/cudnn/backend/v9.3.0/developer/core-concepts.html>
- [3] Huang, H., Hu, X., Li, X., & Xiong, X. (2021). An efficient loop tiling framework for convolutional neural network inference accelerators. *IET Circuits Devices & Systems*, 16(1), 116–123. <https://doi.org/10.1049/cds2.12091>
- [4] *Better performance with the tf.data API.* (n.d.). TensorFlow.  
[https://www.tensorflow.org/guide/data\\_performance](https://www.tensorflow.org/guide/data_performance)
- [5] *Loop interchange | Code Guidelines for Correctness, Modernization, Security, Portability, and Optimization.* (2025, August 22).  
<https://open-catalog.codee.com/Glossary/Loop-interchange/>